

Ginansya — Documentation

An offline compound-interest learning app for Android, written in Java. Two screens: a Home dashboard summarizing the latest projection, and a Calculator that takes the inputs and renders the result with a chart.

1. Compound Interest Logic

The formula

The calculator uses the standard compound interest formula extended with a recurring deposit added at the end of each compounding period:

$$A = P(1 + r/n)^{nt} + PMT \times [((1 + r/n)^{nt} - 1) / (r/n)]$$

Symbol	Meaning
A	Future value (final balance)
P	Principal — starting amount
r	Annual interest rate (as a decimal)
n	Compoundings per year (1, 2, 4, 12, or 365)
t	Time in years (decimals allowed)
PMT	Deposit added per compounding period

Implementation

`CompoundInterestCalculator.project()` doesn't compute the closed form. It iterates period-by-period so that the running balance series is also produced — that's what feeds the chart.

```
i = r / n
N = max(1, round(t * n))
balance = P
for p from 1 to N:
    balance = balance * (1 + i) + PMT
```

After N periods, `balance` is the future value, `P + PMT × N` is the total contributed, and the difference is the interest earned. The list of balances (one per period plus the starting point) is what the chart renders.

Why iterate

The chart needs every period's running balance, not just the endpoint, so the closed form wouldn't save anything. Iterating once is $O(N)$, and N maxes out at a few thousand even for daily compounding over decades — negligible cost on every keystroke.

2. App Architecture

Two-screen flow

```
Home (DashboardFragment)
  | "Open calculator" button
  v
Calculator (CalculatorFragment)
```

```
| back arrow / system back
v
Home (re-reads state on resume)
```

State flow

The Calculator persists state to SharedPreferences on every keystroke (apply(), async). When Home becomes visible (onResume), it reads the saved values, runs them through the formula, and updates its three TextViews and the insight card.

```
User types in Calculator
v
recalc() reads the inputs
v
CalculatorState.update(...) writes prefs + computes Result
v
Calculator updates its UI
v
(user navigates back to Home)
v
DashboardFragment.onResume()
v
CalculatorState.getResult() reads prefs + recomputes Result
v
Home updates its UI
```

No observer pattern, no LiveData. Home and Calculator are never visible at the same time, so writing on every change and reading on visibility is sufficient.

3. Codebase Layout

Packages

Package	Purpose
com.example.ginansya	Activity entry point
com.example.ginansya.data	Persistence
com.example.ginansya.domain	Pure logic (the formula)
com.example.ginansya.util	Formatting helpers
com.example.ginansya.ui.dashboard	Home fragment
com.example.ginansya.ui.calculator	Calculator fragment

File responsibilities

File	Responsibility
MainActivity.java	Inflates the nav host, applies status-bar (top) and nav-bar (bottom) insets. No business logic.
data/CalculatorState.java	Static API over SharedPreferences. update() writes, getters read, getResult() re-runs the formula.
domain/CompoundInterestCalculator.java	Pure function. Five inputs, returns Result with future value, contributed, interest, and the total.
util/CurrencyUtils.java	Locale en-PH with peso symbol. format() for amounts, formatCompact() for chart axis labels.
ui/dashboard/DashboardFragment.java	Reads state in onResume, populates the breakdown TextViews, derives the insight text, updates the insight card.
ui/calculator/CalculatorFragment.java	Holds the inputs, prefills from state, attaches text watchers that recalc on every change.

Persistence schema

SharedPreferences file calculator_state in the app's private directory.

Key	Stored as	Notes
principal	String	Parsed as double
monthly	String	Legacy key name; semantics now "deposit per period"
rate	String	Annual rate as percent
years	String	Decimals allowed (e.g. 7.5)
compounds_per_year	int	One of {1, 2, 4, 12, 365}

4. Build & Run

From the project root:

```
./gradlew.bat assembleDebug
```

Open in Android Studio and run on a device or emulator. The first build pulls MPAndroidChart from JitPack (around two minutes).